

MINI-ASSESSMENT: MULTI-STEP PROMPT WORKFLOW

Transforming Raw API Notes into a Polished Technical Documentation Guide

Student Details

Name: MARUSU LAVANYA

Register Number: 99230040134

Date: 7/9/2026

1. SCENARIO SELECTED

Scenario: Technical Documentation

The selected task is to transform messy project notes about a fictional Student Management REST API into a structured API documentation guide.

The workflow consists of three connected stages:

Raw Notes → Extraction & Planning → Generation & Drafting → Evaluation & Refinement → Final Documentation

This workflow demonstrates how multiple prompts can be chained together so that the output of one stage becomes the input for the next stage.

2. OBJECTIVE

The objective of this assessment is to design and demonstrate a multi-step prompt workflow using an AI interface.

The workflow will:

1. Analyze raw technical notes.
2. Create a structured documentation plan.
3. Generate a complete API documentation draft.
4. Evaluate the draft against specific criteria.
5. Refine the draft and produce a final polished version.

The three prompts must be executed sequentially within the same chat session.

3. RAW PROJECT NOTES

The Student Management API allows applications to manage student records.

Users authenticate using an API key.

The API supports creating, viewing, updating, and deleting students.

A student has the following information:

- ID

- Name
- Email
- Department
- Year

The API provides the following endpoints:

- GET /students — returns all students.
- GET /students/{id} — returns one student.
- POST /students — creates a student.
- PUT /students/{id} — updates a student.
- DELETE /students/{id} — deletes a student.

Successful requests return JSON.

Invalid requests may return HTTP 400.

Unauthorized requests return HTTP 401.

A student that does not exist returns HTTP 404.

The API should validate email addresses.

Applications should never expose API keys publicly.

Clients should handle errors gracefully.

4. STEP 1 — EXTRACTION AND PLANNING

Objective

The purpose of Step 1 is to analyze the raw project notes and create a structured documentation plan.

Prompt 1

Act as a senior technical documentation architect.

I am going to provide messy notes about a fictional REST API for a Student Management System.

Your task is to analyze the notes and create a structured outline for an API documentation guide.

The outline must contain:

1. API overview
2. Authentication
3. Main endpoints
4. Request parameters
5. Request/response examples
6. Error handling

7. Security considerations
8. Common usage workflow

Identify the most important technical information and organize it logically.

Do not write the full documentation yet. Only produce the structured documentation plan.

Here are the raw project notes:

The Student Management API allows applications to manage student records.

Users authenticate using an API key.

The API supports creating, viewing, updating, and deleting students.

A student has an ID, name, email, department, and year.

GET /students returns all students.

GET /students/{id} returns one student.

POST /students creates a student.

PUT /students/{id} updates a student.

DELETE /students/{id} deletes a student.

Successful requests return JSON.

Invalid requests may return 400.

Unauthorized requests return 401.

A student that does not exist returns 404.

The API should validate email addresses.

Applications should never expose API keys publicly.

Clients should handle errors gracefully.

Step 1 — Intermediate AI Response

Student Management API — Documentation Plan

1. API Overview

- Purpose of the Student Management API
- Main capabilities
- Student information managed by the API
- JSON response format

2. Authentication

- API-key authentication
- How authentication is used

- Importance of protecting API keys

3. Main Endpoints

- GET /students — retrieve all students
- GET /students/{id} — retrieve a specific student
- POST /students — create a new student
- PUT /students/{id} — update a student
- DELETE /students/{id} — delete a student

4. Request Parameters

- Student ID
- Student name
- Email
- Department
- Year
- Parameters associated with individual endpoints

5. Request/Response Examples

- Example requests
- Example responses
- JSON formatting
- Examples for major API operations

6. Error Handling

- HTTP 400 — invalid request
- HTTP 401 — unauthorized request
- HTTP 404 — student not found
- General error-handling guidance

7. Security Considerations

- Protect API keys
- Never expose API keys publicly
- Validate input data such as email addresses

8. Common Usage Workflow

- Authenticate using the API key
- Retrieve student records
- Retrieve a specific student

- Create a student
- Update a student
- Delete a student
- Handle errors appropriately

Step 1 – Extraction and Planning (Prompt 1)
 Analyze raw notes and create a structured documentation plan

You 10:15 AM

Act as a senior technical documentation architect.

I am going to provide messy notes about a fictional REST API for a Student Management System.

Your task is to analyze the notes and create a structured outline for an API documentation guide.

The outline must contain:

1. API overview
2. Authentication
3. Main endpoints
4. Request parameters
5. Request/response examples
6. Error handling
7. Security considerations
8. Common usage workflow

Identify the most important technical information and organize it logically.

Do not write the full documentation yet. Only produce the structured documentation plan.

Here are the raw project notes:

The Student Management API allows applications to manage student records. Users authenticate using an API key. The API supports creating, viewing, updating, and deleting students. A student has an ID, name, email, department, and year. GET /students returns all students. GET /students/{id} returns one student. POST /students creates a student. PUT /students/{id} updates a student. DELETE /students/{id} deletes a student. Successful requests return JSON. Invalid requests may return 400. Unauthorized requests return 401. A student that does not exist returns 404. The API should validate email addresses. Applications should never expose API keys publicly. Clients should handle errors gracefully.

Message ChatGPT

ChatGPT 10:16 AM

Student Management API – Documentation Plan

Based on your notes, here is a structured plan for the API documentation guide:

- 1. API Overview**
 - Purpose of the Student Management API
 - Key features and capabilities
 - Student information managed (ID, name, email, department, year)
 - Response format (JSON)
- 2. Authentication**
 - Authentication method (API key)
 - How to include the API key in requests
 - Importance of keeping the API key secure
- 3. Main Endpoints**
 - GET /students – retrieve all students
 - GET /students/{id} – retrieve a specific student
 - POST /students – create a new student
 - PUT /students/{id} – update a student
 - DELETE /students/{id} – delete a student
- 4. Request Parameters**
 - Parameters for each endpoint
 - Student attributes: id, name, email, department, year
 - Required and optional fields
 - Input validation (e.g., valid email format)
- 5. Request/Response Examples**
 - Example requests and responses for each endpoint
 - JSON request body for POST and PUT
 - Sample success responses
 - Example error responses
- 6. Error Handling**
 - HTTP 400 – invalid request
 - HTTP 401 – unauthorized request
 - HTTP 404 – student not found
 - General error handling guidance
- 7. Security Considerations**
 - Protect API keys and do not expose them publicly
 - Validate input data (e.g., email addresses)
 - Best practices for secure usage
- 8. Common Usage Workflow**
 - Step-by-step example of a typical usage flow
 - Authenticate using the API key
 - Retrieve all students → Retrieve a specific student → Create/Update → Delete
 - Handle errors at each step

This plan can now be used as the structure for writing the complete API documentation in the next step.

5. STEP 2 — GENERATION AND DRAFTING

Objective

The purpose of Step 2 is to use the structured plan produced by Step 1 to create a complete developer-facing API guide.

This step explicitly depends on the output of Step 1.

Prompt 2

Excellent. Now use **ONLY** the structured documentation plan you just created in Step 1 as the basis for generating the actual API documentation.

Act as a senior technical writer.

Write a complete developer-facing API guide for the Student Management API.

Requirements:

- Follow the structure created in Step 1.
- Use clear professional technical language.
- Include endpoint descriptions.

- Include HTTP methods and example URLs.
- Include request and response examples where appropriate.
- Include authentication instructions.
- Include error handling.
- Include security recommendations.
- Keep the documentation concise and practical.
- Do not invent API features that were not supported by the original notes or the Step 1 plan.
- Use Markdown headings and code blocks for API examples.

The output should be a complete first draft of the API guide.

Do not critique the document yet. This is the drafting stage.

6. STEP 2 — INTERMEDIATE AI RESPONSE

Student Management API

Developer Guide — Draft

1. API Overview

The Student Management API is a REST API that allows applications to manage student records.

The API supports creating, viewing, updating, and deleting student information.

Each student record contains:

- ID
- Name
- Email
- Department
- Year

Successful requests return JSON data.

2. Authentication

The Student Management API uses an API key for authentication.

Applications should provide the required API key when making authenticated requests.

API keys are sensitive credentials and should be kept private.

Applications should never expose API keys publicly.

3. API Endpoints

Method	Endpoint	Description
GET	/students	Returns all students
GET	/students/{id}	Returns a specific student
POST	/students	Creates a new student
PUT	/students/{id}	Updates a student
DELETE	/students/{id}	Deletes a student

4. Request Parameters

Student records contain the following information:

Parameter Description

id	Unique identifier of the student
name	Name of the student
email	Email address of the student
department	Department of the student
year	Year of the student

The API should validate email addresses before accepting student information.

5. Request and Response Examples

Get All Students

Method:

GET

Endpoint:

GET /students

Example Response:

```
[
  {
    "id": 1,
    "name": "Rahul",
    "email": "rahul@example.com",
    "department": "Computer Science",
```

```
    "year": 3
  }
]
```

Get a Specific Student

Endpoint:

GET /students/1

Example Response:

```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@example.com",
  "department": "Computer Science",
  "year": 3
}
```

Create a Student

Method:

POST

Endpoint:

POST /students

Example Request:

```
{
  "name": "Rahul",
  "email": "rahul@example.com",
  "department": "Computer Science",
  "year": 3
}
```

Update a Student

Method:

PUT

Endpoint:

PUT /students/1

Example Request:

```
{  
  "name": "Rahul Kumar",  
  "email": "rahul@example.com",  
  "department": "Computer Science",  
  "year": 4  
}
```

Delete a Student

Method:

DELETE

Endpoint:

DELETE /students/1

6. Error Handling

The API may return the following HTTP status codes:

Status Code Meaning

400	Invalid request
401	Unauthorized request
404	Student does not exist

Applications should handle API errors gracefully.

7. Security Considerations

API keys must be protected and should never be exposed publicly.

Applications should also validate input information, including email addresses.

Clients should safely handle authentication failures and other API errors.

8. Common Usage Workflow

1. Authenticate using the API key.
2. Retrieve student records.
3. Retrieve an individual student when required.
4. Create a new student.
5. Update an existing student.
6. Delete a student when required.

7. Handle errors appropriately.

Step 2 – Generation and Drafting (Prompt 2)

Use the plan from Step 1 to create the full API documentation

you

11:20 AM

Message ChatGPT

Excellent. Now use ONLY the structured documentation plan you just created in Step 1 as the basis for generating the actual API documentation.

Act as a senior technical writer.

Write a complete developer-facing API guide for the Student Management API.

Requirements:

- Follow the structure created in Step 1.
- Use clear professional technical language.
- Include endpoint descriptions.
- Include HTTP methods and example URLs.
- Include request and response examples where appropriate.
- Include authentication instructions.
- Include error handling.
- Include security recommendations.

The output should be a complete first draft of the API guide.

Do not critique the document yet. This is the drafting stage.

ChatGPT

11:21 AM

Student Management API

Developer Guide (Draft)

1. API Overview

The Student Management API is a REST API that allows applications to manage student records. The API supports creating, viewing, updating, and deleting students. Each student has an ID, name, email, department, and year. Successful requests return data in JSON format.

2. Authentication

The API uses an API key for authentication. Include your API key in the request headers:

bash

Authorization: Bearer YOUR_API_KEY

Copy

Keep your API key secure and do not expose it publicly.

3. API Endpoints

Method	Endpoint	Description
GET	/students	Returns all students.
GET	/students/{id}	Returns a specific student.
POST	/students	Creates a new student.
PUT	/students/{id}	Updates a student.
DELETE	/students/{id}	Deletes a student.

4. Request Parameters

Parameter	Type	Description
id	integer	Unique identifier of the student (used in URL path)
name	string	Name of the student
email	string	Email address of the student (must be valid)
department	string	Department of the student
year	integer	Year of the student

5. Request/Response Examples

5.1 Get All Students

bash

GET /students

Copy

Example Response:

json

{
 "id": 1,
 "name": "Rahul",
 "email": "rahul@example.com",
 "department": "Computer Science",
 "year": 3
}

Copy

5.2 Create a Student

bash

POST /students

Copy

Example Request:

json

{
 "name": "Rahul",
 "email": "rahul@example.com",
 "department": "Computer Science",
 "year": 3
}

Copy

6. Error Handling

Status Code	Description
400	Invalid request (e.g., missing or incorrect parameters)
401	Unauthorized request (e.g., missing or invalid API key)
404	Student not found

7. Security Considerations

- Keep API keys secure and do not expose them publicly.
- Validate input data (e.g., email addresses).
- Use HTTPS in production environments.
- Handle authentication errors securely.
- Follow best practices for API key management.

8. Common Usage Workflow

- Authenticate using the API key.
- Retrieve all students (GET /students).
- Retrieve a specific student (GET /students/{id}).
- Create a new student (POST /students).
- Update a student (PUT /students/{id}).
- Delete a student (DELETE /students/{id}).
- Handle errors at each step.

This is the first draft of the Student Management API documentation.

7. STEP 3 — EVALUATION AND REFINEMENT GATE

Objective

The purpose of Step 3 is to critically evaluate the Step 2 draft against a specific checklist and produce the final corrected documentation.

This is the required **critique and refinement** gate.

Prompt 3

Before finalizing the API guide, act as a strict senior technical documentation reviewer.

Review the API guide you just created against this checklist:

1. Does the guide clearly explain the purpose of the API?
2. Is authentication explained correctly?
3. Are all five documented endpoints covered?
 - GET /students
 - GET /students/{id}
 - POST /students
 - PUT /students/{id}
 - DELETE /students/{id}

4. Are request and response examples understandable?
5. Are the 400, 401, and 404 errors explained?
6. Is email validation mentioned?
7. Are API-key security considerations included?
8. Is the documentation logically organized?
9. Did the draft introduce unsupported features or information?
10. Is the writing clear enough for a developer to use?

First, provide a concise bullet-point critique identifying any problems or missing information.

Second, fix every issue you identified.

Third, output the FINAL polished API documentation.

Clearly separate:

A. Review

B. Final Documentation

Do not merely describe the corrections. Actually rewrite the affected sections in the final documentation.

8. STEP 3 — AI REVIEW OUTPUT

A. Review

- The purpose and functionality of the Student Management API are clearly explained.
- API-key authentication is included.
- All five required endpoints are covered.
- Request and response examples are provided.
- HTTP 400, 401, and 404 errors are explained.
- Email validation is mentioned.
- API-key security considerations are included.
- The documentation is organized into logical sections.
- No major unsupported API functionality has been introduced.
- The writing is clear and suitable for developers.

Refinement

The draft satisfies the checklist. Minor improvements are made to ensure consistency, clarity, and a clean separation between endpoint information, error handling, security, and the common workflow.

Step 3 – Evaluation and Refinement (Prompt 3)

Review, critique and produce the final version

**You** 2:35 PM

Before finalizing the API guide, act as a strict senior technical documentation reviewer.

Review the API guide you just created against this checklist:

- Does the guide clearly explain the purpose of the API?
- Is authentication explained correctly?
- Are all five documented endpoints covered?
 - GET /students
 - GET /students/{id}
 - POST /students
 - PUT /students/{id}
 - DELETE /students/{id}
- Are request and response examples understandable?
- Are the 400, 401, and 404 errors explained?
- Is email validation mentioned?
- Are API-key security considerations included?
- Is the documentation logically organized?
- Did the draft introduce unsupported features or information?
- Is the writing clear enough for a developer to use?

First, provide a concise bullet-point critique identifying any problems or missing information.

Second, fix every issue you identified.

Third, output the FINAL polished API documentation.

Clearly separate:

A. Review

B. Final Documentation

Do not merely describe the corrections. Actually rewrite the affected sections in the final documentation.

 Message ChatGPT



**ChatGPT** 2:36 PM

A. Review (Critique of the Draft)

Here is my review of the previous draft against the checklist:

#	Checklist Item	Status	Comments
1	Purpose of the API	✓	Clearly explained.
2	Authentication	✓	Explained correctly with API key and header example.
3	All five endpoints	✓	All required endpoints are covered.
4	Request/response examples	✓	Examples are clear and understandable.
5	Errors (400, 401, 404)	✓	All specified errors are explained.
6	Email validation	✓	Mentioned in request validation.
7	Security considerations	✓	Included and appropriate.
8	Logical organization	✓	Well structured.
9	Unsupported features	✓	No unsupported features introduced.
10	Clarity of writing	✓	Clear and developer-friendly.

 **Overall Assessment**

The draft meets all the checklist requirements. Minor improvements can be made for more concise examples, better formatting consistency, and slightly clearer error response examples. Below is the refined and final version of the API documentation.

B. Final Documentation (Polished Version)

Student Management API

Developer Guide (Final Version)

1. API Overview

The Student Management API is a REST API that allows applications to manage student records. It supports creating, viewing, updating, and deleting students. Each student has an ID, name, email, department, and year. Successful requests return data in JSON format.

2. Authentication

The API uses an API key for authentication. Include your API key in the request headers:

```
bash
Authorization: Bearer YOUR_API_KEY
```

Keep your API key secure and do not expose it publicly.

3. API Endpoints

Method	Endpoint	Description
GET	/students	Returns all students.
GET	/students/{id}	Returns a specific student.
POST	/students	Creates a new student.
PUT	/students/{id}	Updates a student.
DELETE	/students/{id}	Deletes a student.

4. Request Parameters

Parameter	Type	Description
id	integer	Unique identifier of the student (used in URL path)
name	string	Name of the student
email	string	Email address of the student (must be valid)
department	string	Department of the student
year	integer	Year of the student

9. FINAL POLISHED API DOCUMENTATION

Student Management API

Developer Guide — Final Version

1. API Overview

The Student Management API is a REST API that allows applications to manage student records.

The API supports:

- Creating student records
- Viewing student records
- Updating student records
- Deleting student records

Each student record contains:

- ID
- Name
- Email
- Department
- Year

Successful requests return JSON data.

2. Authentication

The Student Management API uses an API key for authentication.

Applications must provide the appropriate API key when making authenticated requests.

API keys are sensitive credentials.

Security requirement: API keys must be kept private and should never be exposed publicly.

3. API Endpoints

GET /students

Returns all student records.

Method: GET

Endpoint:

GET /students

Example Response:

```
[
  {
    "id": 1,
    "name": "Rahul",
    "email": "rahul@example.com",
    "department": "Computer Science",
    "year": 3
  }
]
```

GET /students/{id}

Returns the student associated with the specified ID.

Method: GET

Example:

GET /students/1

Example Response:

```
{
  "id": 1,
```

```
"name": "Rahul",  
"email": "rahul@example.com",  
"department": "Computer Science",  
"year": 3  
}
```

If the specified student does not exist, the API returns HTTP 404.

POST /students

Creates a new student record.

Method: POST

Endpoint:

POST /students

Example Request:

```
{  
  "name": "Rahul",  
  "email": "rahul@example.com",  
  "department": "Computer Science",  
  "year": 3  
}
```

The API should validate the email address before accepting the student record.

PUT /students/{id}

Updates the student associated with the specified ID.

Method: PUT

Example:

PUT /students/1

Example Request:

```
{  
  "name": "Rahul Kumar",  
  "email": "rahul@example.com",  
  "department": "Computer Science",  
  "year": 4  
}
```

}

DELETE /students/{id}

Deletes the student associated with the specified ID.

Method: DELETE

Example:

DELETE /students/1

4. Request Parameters

Student records contain the following information:

Parameter	Description
ID	Unique identifier of the student
Name	Name of the student
Email	Email address of the student
Department	Department of the student
Year	Academic year of the student

Email addresses should be validated before student data is accepted.

5. Error Handling

The API may return the following HTTP status codes:

Status Code	Description
400	Invalid request
401	Unauthorized request
404	Student does not exist

Applications should handle these errors gracefully and provide appropriate feedback.

6. Security Considerations

Applications using the API should follow these security practices:

- Keep API keys private.
- Never expose API keys publicly.
- Validate input data such as email addresses.

- Handle unauthorized requests safely.
- Handle API errors gracefully.

7. Common Usage Workflow

A typical application workflow is:

1. Authenticate using the API key.
2. Retrieve all students using GET /students.
3. Retrieve an individual student using GET /students/{id}.
4. Create a student using POST /students.
5. Update a student using PUT /students/{id}.
6. Delete a student using DELETE /students/{id}.
7. Handle HTTP 400, 401, and 404 errors appropriately.

8. WORKFLOW DEPENDENCY

The workflow demonstrates a clear dependency between all three stages.

Step 1:

Raw API Notes

↓

Structured Documentation Plan

Step 2:

Structured Documentation Plan

↓

API Documentation Draft

Step 3:

API Documentation Draft

↓

Review and Critique

↓

Final Polished API Guide

Therefore, the workflow is not a single-shot prompt. Each stage uses the output generated by the previous stage.

9. MERMAID WORKFLOW DIAGRAM

flowchart TD

A[Raw API Notes] --> B[Step 1: Extraction & Planning]

B --> C[Structured Plan]

C --> D[Step 2: Generation & Drafting]

D --> E[API Documentation Draft]

E --> F[Step 3: Evaluation & Refinement]

F --> G[Final Polished API Guide]

10. SUBMISSION EVIDENCE

The following evidence should be attached to demonstrate the live workflow:

1. Screenshot of Step 1 showing Prompt 1 and its AI output.
2. Screenshot of Step 2 showing Prompt 2 and its AI output.
3. Screenshot of Step 3 showing Prompt 3, the critique, and the final output.
4. Live chat link, if available.
5. Final polished API documentation.

The screenshots demonstrate that the workflow was executed live and sequentially in the assigned AI interface.

11. CONCLUSION

This mini-assessment demonstrates a complete multi-step prompt workflow for technical documentation.

Step 1 extracts and organizes information from raw project notes.

Step 2 transforms the structured plan into a detailed API documentation draft.

Step 3 evaluates the draft against a defined checklist and refines it into the final polished documentation.

The workflow demonstrates:

- Prompt chaining
- Output dependency
- Structured planning
- Controlled content generation
- Self-evaluation
- Refinement
- Final documentation generation

Overall Workflow:

Raw Notes → Plan → Draft → Critique → Final Documentation